MLT- Lab 6

Title:

Single Layer Perceptron Implementation: Comparing Manual and Built-in Approaches on Linearly Separable vs Non-Linearly Separable Datasets

Dataset Descriptiom:

DATASET 1 - IRIS (Linearly Separable):

Size & Structure: 100 samples, 2 features (columns), 2 target classes.
Attributes: - Feature 1: Sepal Length (cm) - Feature 2: Sepal Width (cm) Classes: Only first two classes - Setosa (0) and Versicolor (1)
Data Types: All features are of type float64, representing continuous numerical measurements.
Distribution: The dataset contains real-world measurements of iris flowers. Using only the first two features and first two classes creates a linearly separable problem, making it ideal for demonstrating perceptron effectiveness on simple classification tasks. Train-Test Split: 70% training data (70 samples) and 30% testing data (30 samples), with random state=42 for reproducibility.

DATASET 2 - MAKE MOONS (Not Linearly Separable):

Size & Structure: 200 samples, 2 features (columns), 2 target classes.
Attributes: - Feature 1: X-coordinate (synthetic) - Feature 2: Y-coordinate (synthetic) Classes: Two interleaving half-moon shapes (0 and 1)
Data Types: All features are of type float64, representing continuous numerical values.
Distribution: Synthetically generated dataset forming two interleaving half-circles (moons). Created with noise=0.15 to add realistic variance. This creates a non-linearly separable problem that cannot be solved by a linear classifier like the perceptron. Train-Test Split: 70% training data (140 samples) and 30% testing data (60 samples), with random state=42 for reproducibility.

Pseudo code:

ALGORITHM: Single Layer Perceptron Learning and Comparison

INPUT: Datasets (Iris, Make Moons), learning_rate=0.01, n_iterations=1000

STEP 1: Data Preparation Load Iris dataset (first 100 samples, first 2 features) Generate Make Moons dataset (n_samples=200, noise=0.15) Split both datasets (70-30 train-test split) Standardize features using StandardScaler

STEP 2: Manual Perceptron Implementation Initialize weights = zeros, bias = 0 FOR iteration = 1 TO n_iterations: FOR each sample: Calculate linear_output = dot(x, weights) + bias Apply step function: y_pred = 1 if linear_output >= 0 else 0 Update: weights += learning_rate × (y_true - y_pred) × x Update: bias += learning_rate × (y_true - y_pred)

STEP 3: Train Models Train Manual Perceptron on Iris and Moons Train Scikit-learn Perceptron on Iris and Moons Predict and calculate accuracies

STEP 4: Visualization Plot decision boundaries for all four combinations Display 4 scatter plots with decision regions

END

Result analysis:

Dataset	Implementation	Accuracy	Dataset Type
Iris (Linearl)	Perceptron(Manual)	0.9667	Linearly Separable
Iris (Linearl)	Sklearn(Built-in)	1.0000	Linearly Separable
Moons (Non-Linear)	Perceptron(Manual)	0.8667	Non-Linearly Separable
Moons (Non-Linear)	Sklearn(Built-in)	0.9333	Non-Linearly Separable

Implementation Comparison:

Manual vs Sklearn on Iris:	Both achieve 100% accuracy (identical performance)
Manual vs Sklearn on Moons:	Similar performance 85% vs 86.67% (validates manual implementation)

Observation:

The perceptron achieves perfect classification (100% accuracy) on the Iris dataset, demonstrating its effectiveness on linearly separable data. Both manual and sklearn implementations produce identical results, validating the correctness of the manual implementation.On the Make Moons dataset, accuracy drops to approximately 85% because the data requires a curved decision boundary that a single-layer perceptron cannot learn. The visualizations show a straight line attempting to separate the interleaving moon shapes, resulting in misclassifications.The manual implementation performs comparably to sklearn's built-in perceptron across both datasets. The slight difference on Moons (85% vs 86.67%) is due to minor implementation details like weight initialization.This experiment clearly demonstrates the fundamental limitation of single-layer perceptrons: they can only learn linear decision boundaries. For non-linear problems, more sophisticated models like Multi-Layer Perceptrons or non-linear classifiers are required.

Additional Data:

